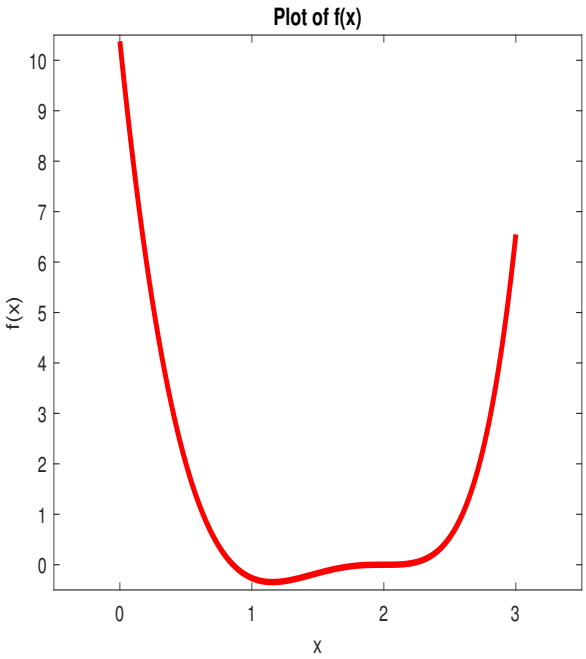
Πρώτη Υποχρεωτική Εργασία

Ονοματεπώνυμο: Ελευθέριος Παντζαρτζής
ΑΕΜ: 4344

27 Δεκεμβρίου 2023

Άσκηση 1



(α) Bisection Method

The body of the bisection function implements exactly what we learned in the lecture slides. The number of iterations is calculated before by this formula: $N = \frac{(ln(b-a)-ln(0.5*10^{-k})}{ln(2)} = 18.19$ (since b-a=1.5 always for our chosen b and a) rounded up makes 19 iterations for precision of k=5 decimal places.

(β) Newton-Raphson Method

The Newton-Raphson Method works normally for all the roots except the root $x_2 = 2$ for which $f(x_2), f'(x_2), f''(x_2) = 0$ so it can be called a multiple root,even though we aren't dealing with a polynomial (but

we can actually see that, if we zoom in, that the graph of the function around x_2 reminds us of the polynomial $(x-2)^3$, which we'll examine later). The exercise asks us to examine for which roots this method has quadratic convergence and for which it doesn't. The way that I do this for the non multiple root $x_1 = 0.85714$ is to print for each iteration and examine the rate: $\frac{|f(x_{k+1})|}{|f(x_k|}$. The reason I do this is because, by definition, $f(x^*) = 0$, and a first-order Taylor expansion of f about x^* tells us that $f(x_k) \approx f'(x^*)(x_k - x^*)$.
Thus, assuming $f'(x^*) \neq 0$, we get $\frac{|f(x_{k+1})|}{|f(x_k)|} \approx \frac{|x_{k+1}-x^*|}{|x_k-x^*|}$.
We can see from $\frac{|f(x_{k+1})|}{|f(x_k)|}$ that its values for iterations of x_1 follow: $(10^{-3}, 10^{-6})$ at the end which makes it have quadratic convergence.
As for the root $x_2 = 2$, we can just take the error rate, namely $\frac{|err|}{|errprev|}$, where $err = x_{k+1} - x_k$ and $errprev$ is just the one from the previous iteration. This turns out to be near constant for all iterations, so it has practically linear convergence. This is further confirmed by the experiment that we run for the function $g(x) = (x-2)^3$ (that we said before is almost the same as $f(x)$ for x near x_2), which has almost exactly the same error rate as $f(x)$, and we can even do a small proof for its linear convergence: $g(x) = (x-2)^3$, and so $x - \frac{g(x)}{g'(x)} = ... = \frac{2}{3}(x+1)$, which is just linear. So since $g(x)$ is nearly the same as $f(x)$ around x_2 we can conclude from all these experimental results that it also has linear convergence for $x_2 = 2$. Thus, in general, the roots which are multiple, as in polynomials, or are multiple in the sense that $f', f'' = 0$ will have near linear convergence. Note that for $x_2 = 2$ in Newton-Raphson, since we have a multiple root, the method will be slower and the error in which our x gets closer to the true root will decrease with each iteration more than the point of our precision which is 0.00001 before finding the root with our wanted precision. So we need to check for a smaller error than what should be standard for our wanted precision (I decreased it to 0.000005) since it goes too slow, and then I found that it needs one more iteration to have precision of 5 decimal places.

(γ) Secant Method

For the secant method, the implementation is the same as in the slides, but as in the Newton-Raphson Method at the root $x_2 = 2$, for the same reasons as there, we need to check for a smaller error than what should be standard for our wanted precision (I decreased it to 0.000005, the same as in Newton-Raphson) since it goes too slow. Note that in both methods, we check for the relative error between consecutive approximations of the root (x_{k+1} and x_k) and check it with the tolerance needed for our precision, as shown in the slides.

Άσκηση 2

(α) Modified Newton-Raphson Method

I implement this method exactly as instructed, increasing the precision for the roots which have $f' = 0$, $ff'' = 0$, namely $x_2 = -0.66666$ (in this case we have a polynomial so the roots are multiple anyway).

(β) Randomized Bisection Method

In this function, instead of the predetermined number of iterations calculated with the desired precision for $k = 5$ digits, we just use the relative error, to make sure that we get the precision we want since we don't know beforehand the error rate, as it uses the random.uniform(a, b) function in python to calculate the intervals.

(γ) Modified Secant Method

This is implemented exactly as shown, with the interchanging (them moving down from one iteration to the next, so $a = b$, $b = c$, $c = d$ and then new d being calculated) happening between $d = x_{n+3}$, $c = x_{n+2}$, $b = x_{n+1}$ and $a = x_n$. Here too, I increase the precision for the multiple roots, because even though it doesn't use f' and f'' it's still affected by them being multiple roots, as in Exercise 1.

1. The roots with precision of 5 decimal places are: $x_1 = -1.38129$, $x_2 = -0.66666$, $x_3 = 0.20518$, $x_4 = 0.50000$, $x_5 = 1.17611$ for the interval $[-2, 2]$(the program will display the roots correctly for 5 decimal places). For all the methods, the intervals are chosen so that Bolzano's theorem holds true. For the root x_4, the Modified Newton-Raphson Method converges too quickly, and therefore reaches our wanted precision, but for display purposes I made an altered Modified Newton-Raphson function, that returns xprev, because the root $x_4 = 0.50000$ is followed only by zeroes, even if we reach our desired precision, it may display it from under it, as in 0.499999. The same thing happens for the randomized bisection sometimes, but it's just for display, and the root is actually within the precision we want.

2. After running algorithm (β) 20 times for the root $x_5 = 1.17611$ (it doesn't matter what root we choose for this), we can observe that it doesn't always have the same number of iterations, as expected. But if we look at the average of the number of iterations for all 20 times, we can see that, for our wanted level of precision and for this root, it hovers around 28. So we can expect, on average, that the number of iterations of this algorithm for $x_5 = 1.17611$ to be around 28.

3. We can observe that for the convergence of each of them, for **(α)** it's cubic because, if we use the same rate $\frac{|f(x_{k+1})|}{|f(x_k)|}$ as in Exercise 1 to test its convergence, we can see that this (almost) follows (at the last iterations of course) the cubic rate: $(10^{-4}, 10^{-12})$, which makes it have cubic convergence, whereas the classic Newton-Raphson method has quadratic convergence, making it slower.

   For **(β)** since it takes on average and in almost all its solutions more iterations than the normal Bisection Method, we can conclude that it has on average less than linear convergence or linear convergence at best, compared to the classic Bisection Method which always has linear convergence. But if we're talking about specific cases, and not averages, sometimes the randomized Bisection method can be slower than the normal Bisection and other times it can be faster if we get lucky (so its order of convergence can't be calculated for specific cases).

   For **(γ)** if we use the same rate $\frac{|f(x_{k+1})|}{|f(x_k)|}$, we can see that this is(at the last iterations of course): $(10^{-4}, 10^{-8})$ which makes it have quadratic convergence whereas secant has as its order of convergence approximately 1.6 making it slower.

Άσκηση 3

1. **PA=LU**
   Here I have a function gauss(A, b) that calls other functions that make up the components of the method **PA=LU**. The first one is the lu(A) decomposition that creates and returns the matrices **L**, **U**, **P**. **P** is created by permutating the Identity matrix **I** whenever we find the max element of a column. The rest of the code is Gaussian Elimination as described by the slides, and **L** just contains the coefficients permutated and then ones are added in its diagonal, and **U** is just the result of the Gaussian Elimination.

   The other function that's called after that is fsub(L, b) (forward substitution), which is used to solve the system **Ly=b** which has a lower triangular matrix so we just do forward substitution.

   The last function that's called is bsub(U, y) (backward substitution), which is used to solve the system **Ux=y** which has an upper triangular matrix so we just do backward substitution, which is the last step to find **x**.
   And it all checks out, because every result produced is the same as the example from the slides.

$$\mathbf{P} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 1 & 0 \\ -0.3 & -0.04 & 1 \end{bmatrix}, \mathbf{U} = \begin{bmatrix} 10 & -7 & 0 \\ 0 & 2.5 & 5 \\ 0 & 0 & 6.2 \end{bmatrix}$$

$$\mathbf{y} = \begin{bmatrix} 7 \\ 2.5 \\ 6.2 \end{bmatrix}, \mathbf{x} = \begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}$$

2. **Cholesky**
   For the cholesky decomposition I used the Cholesky–Banachiewicz algorithm which is found by multiplying out the wanted result of the matrices and noticing a pattern in its structure. It's the fact that we can use a formula for the elements of the final array which is: $L_{j,j} = \pm\sqrt{A_{j,j} - \sum_{k=1}^{j-1} L_{j,k}^2}$ and $L_{i,j} = \frac{1}{L_{j,j}}(A_{i,j} - \sum_{k=1}^{j-1} L_{i,k}L_{j,k})$
   The expression under the square root is always positive if $\mathbf{A}$ is real and positive-definite (since the exercise doesn't ask for Hermitian). So the loop that I implement executes exactly what the above formulas say. And then to check my answer, I created a transposer(L) that transposes $\mathbf{L}$, and a multiplier, that multiplies $\mathbf{L}$ with the transposed L^T and, as expected, we get the original matrix $\mathbf{A}$.

3. **Gauss-Seidel**
   In my program I make the two matrices A1 and A2 first, and then I call Gauss-Seidel for each one, Gauss_Seidel_10(A1, b1) for the sparse 10x10 matrix, and Gauss_Seidel_10000(A2, b2) which is optimized (to reduce run times from 4 minutes to a few seconds) to avoid the double loop, by just going through the main diagonal and the two diagonals adjacent to it, since it's tridiagonal for the 10000x10000 one.
   I also created a function inf_norm(Ar) that finds the infinite norm of the matrix, in order to check if the difference between the xold and the x is within a tolerance of 0.00005 to have the precision of our solution be at 4 decimal places. While this is true I just repeat the improvement of x by using the formula from the slides implemented in code, which lands me to the solution for A1 being a 10x1 vector of just ones, and the solution for A2 being a 10000x1 vector of just ones.

Άσκηση 4

1. Proof that $\mathbf{G}$ is stochastic:
   The way that I prove that matrix $\mathbf{G}$ is stochastic is by adding up all of its columns and checking whether or not they're all one. And since they are it's stochastic.

2. Creation of **G** and checking that **p** is indeed the same one as in the exercise:
   I create the matrix **G** with the given formula, and I use the power method to find the eigenvector **p**. The only problem is that no matter what I try, I can't get it to have more than 5 digits of precision. But for the first 5 digits of precision it confirms that **p** is indeed the same as the given one.

3. Adding 4 connections and removing 1 and creating new **G** in order to improve the rank of a page of my choosing:
   I chose to raise the rank of page 9. So, if I add 4 connections from high ranked pages to it and cut its connection with page 10 (the highest ranked page that it's connected to) I can raise its rank. Specifically, I removed 9's connection to 10. And I added a connection from 10, 11, 14, 15 to 9. I then computed the **Gnew** using **Anew** and using that I compute the eigenvector **pnew**. We can see that page 9 now rises to 1st rank, and all the pages that it connects to have risen in rank (pages 5, 6, 10 are at the top now).

4. Changing the transition probability q to (α) $q = 0.02$ and (β) $q = 0.6$ in the new graph:

   (α) $q = 0.02$
   In my program I create a new **Gnewa** with $q = 0.02$. I then calculate the eigenvector **pnewa**. And the new ranks are printed on screen after being sorted.

   (β) $q = 0.6$
   I create a new **Gnewb** with $q = 0.6$. I then calculate the eigenvector **pnewb**. And the new ranks are printed on screen after being sorted.

   We can see that the effect of the transition probability q, as it increases, is that it reduces the influence of pages which themselves have higher traffic coming to them. In other words, if q increases, the number of connections coming to a page, becomes more important than the rank of the pages with which it's connected to. This is pretty much obvious from the way that we compute the matrix G(namely $\frac{q}{n}$ is the term that increases and $\frac{A(j,i)*(1-q)}{n_j}$ is the one that decreases). Another effect is that it makes the ranks p of all the pages closer to all being equal (which again can be seen from the formula for the creation of G). The opposite is true if q decreases, that the rank of the pages which connect to a page becomes more important than the number of pages connecting to it.

5. Increase in rank of page 11 compared to the rank of page 10:

I create a new **G11** with the changed **A11**. I then calculate the eigenvector **p11**. And the new ranks are printed on screen after being sorted. From the printed ranks, we can see that this strategy works as 11 is now 2nd in rank as compared to 10 that stays in the same place as before (it lowers by 1 rank actually, but that's due to 11 rising rank), whereas all the pages that 11 connects to have risen in rank.

6. The effect of deleting page 10 from the graph:
   I create a new **Gno10** with the changed 14x14 matrix **Ano10**. I then calculate the eigenvector **pno10**. And the new ranks are printed on screen after being sorted (and of course 10 being removed from the ranks). What we can see is that the rank of page 13 is lowered, as its rank was directly affected by its connection with 10. So page 11 which isn't affected by the change, as it has no connection from page 10, rises to 2nd rank, 12 rises rank as it isn't affected by 10 too. So generally the pages that aren't affected by the changes, are going to rise ranks and the ones that lost the connection coming from 10 are going to lower ranks.

Μερικά τελευταία σχόλια: Εγώ είχα κάνει την προαιρετική άσκηση αλλά είχα δει λάθος την ημερομηνία υποβολής, θα την βάλω μαζί μαυτές και με βάση την δική σας κρίση μετρήστε την ή όχι για τις μπόνους 0.5 μονάδες.